# TypeScript Compiler Architecture — Complete Notes

The two diagrams show the same TypeScript compilation process, but at different levels of detail. The second image introduces the main stages, while the first expands the internal work performed by the compiler.

I'll explain each stage in technical but easy-to-understand language, in the correct order, with examples you can use in your TypeScript learning notes.

## 1. Overall flow of TypeScript compilation

| |
|---|
| **1. TypeScript Source Code**<br>.ts files |
| **2. Lexer**<br>Breaks source code into tokens |
| **3. Parser**<br>Checks syntax and builds an AST |
| **4. Binder**<br>Connects declarations and references |
| **5. Checker**<br>Checks types and semantic correctness |
| **6. Emitter**<br>Generates JavaScript and optional output files |
| **Output**<br>.js, .d.ts, .map (depending on configuration) |

> **Important:** This is a simplified learning model. The real TypeScript compiler can perform some of these activities incrementally or in overlapping ways. The AST is created by the parser, and the binder and checker use it to understand the program.

## 2. Step 1 — TypeScript source code

The process begins with code written by the developer in a .ts file.

Example:

```
let age: number = 21;

function greet(name: string): string {
  return "Hello " + name;
}

console.log(greet("Karan"));
```

Here:

- age is declared as a number.
- name is declared as a string.

- greet() must return a string.
- TypeScript analyzes these declarations before producing JavaScript.

The compiler reads this source code and begins processing its characters and structure.

## 3. Step 2 — Lexer (Scanner)

Purpose: Convert raw source-code characters into meaningful tokens.

The lexer, also called the scanner, reads the code from left to right and identifies individual units such as keywords, identifiers, operators, punctuation and literals.

For example:

```
let age: number = 21;
```

The lexer identifies tokens such as:

| Source text | Token meaning |
|---|---|
| let | Keyword |
| age | Identifier |
| : | Colon punctuation |
| number | Type keyword/name |
| = | Assignment operator |
| 21 | Numeric literal |
| ; | Semicolon punctuation |

Think of the lexer as separating a sentence into words and punctuation. It does not fully understand what the program means; it identifies the individual pieces.

> **Notes:** Lexer = characters → tokens.

## 4. Step 3 — Parser

Purpose: Check the grammatical structure of the tokens and construct an Abstract Syntax Tree (AST).

The parser takes the tokens produced by the lexer and determines how they fit together according to TypeScript's syntax rules.

For example:

```
let age: number = 21;
```

The parser recognizes a variable declaration containing:

- The declaration keyword: let
- The variable name: age
- The type annotation: number
- The initializer: 21

It organizes these into a tree-like structure called an AST.

| Variable Declaration | | |
|---|---|---|
| **age**<br>Identifier | **number**<br>Type annotation | **21**<br>Initializer |

This is a simplified illustration of the AST, not the exact internal node structure.

### What is an AST?

AST stands for Abstract Syntax Tree. It represents source code as structured nodes rather than as a plain string of characters.

### Why is it needed?

- Makes it easier for the compiler to navigate declarations and expressions.
- Helps the binder connect names to their declarations.
- Helps the checker analyze types and expressions.
- Provides the structure used by later compiler stages.

If the code has invalid syntax, such as a missing closing parenthesis, the parser reports a syntax error and may create recovery nodes so that it can continue analyzing other code.

> **Notes:** Parser = tokens → AST + syntax diagnostics.

## 5. Step 4 — Binder

Purpose: Connect declarations with their scope and references, and build the symbol information needed by the checker.

After the parser creates the AST, the binder examines declarations and determines where names are declared and how they relate to their containing scopes.

Consider this code:

```
let age: number = 21;

function greet(name: string) {
  return "Hello " + name;
}
```

The binder helps TypeScript establish information such as:

- age is a variable declared in the current scope.
- greet is a function declaration.
- name is a parameter declared within the function's scope.
- A reference to name inside greet() refers to that parameter, not to an unrelated variable elsewhere.

### What does the binder create?

The diagram shows three important concepts.

### A. Symbol tables

Symbol tables store information about declared names, such as variables, functions, classes and their associated declarations. This helps the compiler resolve identifiers and inspect their

properties.

### B. Parent pointers and scope relationships

The compiler tracks relationships between scopes and declarations. For example, a function's local scope is related to the surrounding scope. This supports lookup of names in enclosing scopes.

### C. Flow nodes

TypeScript maintains control-flow information that helps the checker understand how variables may change type as execution moves through branches, assignments and other statements. The binder contributes to setting up this information.

A useful distinction: the binder establishes symbol and declaration relationships; the checker uses those relationships to perform type analysis. The diagram groups some flow-related information near the binder, but control-flow analysis is not solely the binder's job.

> **Notes:** Binder = AST declarations → symbols, scopes and relationships.

## 6. Step 5 — Checker

Purpose: Check whether the program's types and semantic relationships are valid.

The checker uses the AST and the information established by the binder to determine the types of expressions, verify function calls and identify many kinds of semantic errors.

For example:

```
let age: number = 21;

age = "twenty-one";
```

The checker reports an error because a string is being assigned to a variable declared as a number.

Another example:

```
function add(a: number, b: number): number {
  return a + b;
}

add(10, "20");
```

The call is invalid because the second argument is a string rather than a number.

### What does the checker do?

- Checks whether assigned values are compatible with declared types.
- Checks function arguments and return values.
- Resolves identifiers using symbol information.
- Checks properties and method access on types.
- Performs control-flow-based type narrowing.
- Reports semantic and type errors.

### What are the two boxes shown below the checker in the diagram?

### A. Syntax Check / Short Circuit

This represents checks that can be made without completing every possible type-analysis operation.

For example, if the compiler can already determine that an expression is invalid or that a required check cannot proceed meaningfully, it can report the issue without doing unnecessary work.

The exact phrase “short circuit” is a simplified diagram label, not the name of a separate public TypeScript compiler stage.

### B. Control-flow analysis

The checker tracks which values a variable can have at different points in the program.

```
function printValue(value: string | number) {
  if (typeof value === "string") {
    console.log(value.toUpperCase());
  } else {
    console.log(value.toFixed(2));
  }
}
```

Inside the if branch, TypeScript knows value is a string. In the else branch, it knows value is a number. This is called type narrowing.

### What if there is a type error?

By default, TypeScript can report type errors while still emitting JavaScript, depending on the configuration and the compilation command. For example, noEmitOnError can be enabled to prevent output when errors are present.

Type checking and JavaScript execution are different things: TypeScript types generally do not exist at runtime.

> **Notes:** Checker = symbols + AST + type rules + control flow → diagnostics and type information.

## 7. Step 6 — Emitter

Purpose: Generate output files from the TypeScript source and compiler information.

The emitter uses the source structure and compiler settings to produce JavaScript and, when requested, additional files such as declaration files and source maps.

Consider this TypeScript:

```
function greet(name: string): string {
  return "Hello " + name;
}

console.log(greet("Karan"));
```

A simplified JavaScript output could be:

```
function greet(name) {
  return "Hello " + name;
}

console.log(greet("Karan"));
```

Notice that the type annotations : string have disappeared. JavaScript does not use TypeScript's static type annotations.

The output depends on the compiler settings, such as target, module, declaration and sourceMap.

# 8. The three output files in the diagram

## A. .js — JavaScript file

The JavaScript version of the source code. Browsers and JavaScript runtimes can execute it, subject to their module and runtime requirements.

Example: app.ts → app.js

## B. .d.ts — Declaration file

Describes the types and public API exposed by JavaScript code. Other TypeScript files can use these declarations to understand the available functions, classes, properties and types without needing the original TypeScript implementation.

Example: library.ts → library.d.ts

Enable generation with the declaration compiler option.

## C. .map — Source map

A source map connects locations in generated JavaScript to locations in the original TypeScript source. Developer tools can use it to show the original source when debugging the generated code.

Example: app.js.map

Enable JavaScript source maps with the sourceMap compiler option.

> **Remember:** These files are not all generated every time. The .js output depends on emit settings, while .d.ts and .map files require the relevant options. The diagram shows possible outputs, not mandatory outputs.

# 9. How the stages work together — one complete example

Suppose you write:

```
let score: number = 100;

score = "high";
```

Here's what happens conceptually:

| | | |
|---|---|---|
| **1** | **Lexer** | Recognizes tokens such as let, score, number, 100 and "high". |
| **2** | **Parser** | Builds AST nodes for the variable declaration and assignment. |
| **3** | **Binder** | Establishes the symbol for score and connects the later reference to its declaration. |
| **4** | **Checker** | Detects that "high" is a string and cannot be assigned to score, which has type number. |
| **5** | **Emitter** | May still generate JavaScript despite the type error, unless configuration or the build process prevents emission. |

The key lesson is that the lexer and parser understand the code's tokens and structure, the binder connects names and declarations, and the checker understands their types. The emitter generates the requested output.

## 10. Quick revision table for your notes

| Stage | Main responsibility | Remember it as |
|---|---|---|
| Source code | Developer writes .ts code | Input |
| Lexer | Identifies tokens | Characters → tokens |
| Parser | Builds AST and checks syntax | Tokens → AST |
| Binder | Connects declarations and references | Names → symbols |
| Checker | Checks types and semantics | Symbols + AST → diagnostics |
| Emitter | Generates configured output | Source → output files |

## 11. How to see the compiler in action

You can experiment in your VS Code terminal.

```
npm install -g typescript
tsc --version
```

Create a file named app.ts:

```
let age: number = 21;
console.log(age);
```

Compile it:

```
tsc app.ts
```

TypeScript generates app.js in the same directory under the default settings.

Now change the code to:

```
let age: number = "twenty-one";
```

Run:

```
tsc app.ts
```

You should see a type error. With the default noEmitOnError: false behavior, JavaScript can still be emitted.

## 12. Important interview points

- Is TypeScript compiled to JavaScript? Yes. The TypeScript compiler can transform TypeScript into JavaScript.
- Does JavaScript understand TypeScript types? No. TypeScript type annotations are removed during normal JavaScript emission.
- What is an AST? A tree representation of the program's syntactic structure.
- What is the difference between the binder and checker? The binder establishes symbol and scope relationships; the checker performs type analysis using that information.

- Does TypeScript type checking guarantee runtime safety? No. Static types help catch errors before execution, but they do not automatically validate external data or enforce types at runtime.
- Are all stages completely separate passes? The diagram is a conceptual overview. The actual compiler shares information across stages and can use incremental processing.

**One-line memory trick:**

Lexer → Tokens | Parser → AST | Binder → Symbols | Checker → Types | Emitter → Files